

DataSyncX Copilot Agent

DataSyncX includes a VS Code custom agent for ETL design, routing, implementation, and validation work. The agent lives in the repository so developers who clone DataSyncX can use the same ETL guidance directly from their local workspace.

This agent is the recommended first choice for ETL-writing tasks that may reasonably use DataSyncX, even when the final code change belongs in a product repository instead of the framework itself.

Repository-Local Distribution

The agent is stored under `.github/agents/` inside this repository, so it travels with the repository instead of being tied to one developer's local VS Code profile.

In practice, this means other developers can use the same agent after they:

1. clone or pull the repository revision that contains the `.github` agent files,
2. open the repository folder, or a workspace that includes it, in VS Code,
3. sign in to GitHub Copilot with a VS Code build that supports custom agents.

If the agent does not appear immediately after pull or clone, reload the VS Code window so Copilot refreshes workspace customizations.

Recommended Workspace Shape

For most users, the best setup is not to move product development into the DataSyncX repository.

Instead:

1. keep your product or project repository as the main place where you edit business logic,
2. clone `datasyncx` locally as a separate repository,
3. add both folders to the same VS Code workspace,
4. use the `DataSyncX ETL Agent` to reference DataSyncX docs, demos, abstractions, and framework guidance while you continue implementing product-specific code in your own repository.

Use direct DataSyncX edits only when the change is truly framework-worthy. Keep product predicates, schedules, target schemas, credentials, and business rules in the product repository unless the behavior is clearly reusable.

Adding `datasyncx` to the same VS Code workspace is for agent discovery, documentation lookup, and framework-context review. It does not replace installing `datasyncx` into the product project's Python environment. If the product code imports `datasyncx`, the product environment still needs a real package installation such as `pip install datasyncx` from the appropriate release source. See [installation.md](#).

Use The Agent

1. Clone the DataSyncX repository.
2. Open your product repository in VS Code, or create a workspace that includes your product repository plus the local `datasyncx` clone.
3. Make sure the `datasyncx` folder is part of the active workspace so Copilot can discover the repository-local agent.
4. Open Copilot Chat and select `DataSyncX ETL Agent` from the agent picker.
5. Describe the ETL task, active repo, runtime path, and validation baseline.

The agent definition is stored at `.github/agents/datasyncx-etl.agent.md`.

The durable routing and validation rules for the agent are stored in `etl-agent-playbook.md`. The agent prompt is intentionally thinner than that playbook so DataSyncX-specific operating knowledge can evolve in docs and examples without turning the agent definition into a second copy of the manual.

To keep resumed chats aligned with the latest rules, the agent uses a lightweight version marker at [agent-rules-version.md](#). It probes that marker first and reloads full guidance only when the version tuple changes, which keeps continuity reliable while avoiding unnecessary token-heavy rereads.

This repository-local setup is the recommended sharing model for DataSyncX guidance because the agent can evolve together with framework docs, demos, and validation rules.

If you are working on a product repository that depends on DataSyncX, the normal workflow is: ask design and reuse questions with both repositories open in one workspace, implement product-local changes in the product repository, and edit DataSyncX itself only when the agent and repository evidence both show the change should become reusable framework behavior.

In other words: shared workspace for agent and framework context, package installation for runtime execution.

Good Prompts

Use the agent when you need DataSyncX-specific judgment, such as:

- Decide whether an ETL change belongs in DataSyncX or a product repository.
- Review a reader, writer, processor, task, query-builder, or task-range change.
- Design a batching or runtime-parameter strategy.
- Define the minimum validation gate for a DataSyncX runtime change.
- Implement a bounded DataSyncX change after the scope and validation gate are clear.

Use a generic coding agent only when the work is clearly outside ETL design or clearly unrelated to DataSyncX reuse decisions.

The agent is not limited to template-driven or demo-factory workflows. It can also help with free-style DataSyncX composition where you directly combine readers, writers, processors, task helpers, and utility functions in one script. For the broader pipeline-building options, see [usage.md](#).

For example:

```
Classify this SQL Server writer issue. The product pipeline currently deletes a date slice and
appends replacement rows. Should the fix stay product-local or move into DataSyncX? Use the
current branch and tests to propose the minimum validation gate.
```

```
Create a direct Xeus to PostgreSQL demo script under src/datasyncx/demo by reusing existing
XeusReader and PostgreSQLWriter patterns. Keep it as a copy-first example, do not execute it,
and report the minimum validation gate.
```

```
I do not want a template-generated script. Help me compose a free-style DataSyncX pipeline
that joins one XEUS read with one custom dataframe transform and writes the final dataframe to
PostgreSQL.
```

One concrete direct-demo example now lives at `src/datasyncx/demo/Xeus_to_PostgreSQL.py`. Use it when you want a simple XEUS-to-PostgreSQL baseline that can be copied and adapted without going through demo-factory generation.

Form-Style Prompt Template

Use this fill-in template when you want the `DataSyncX ETL Agent` to quickly build a source-to-target Python sync flow.

For the improvement rationale and rollout checkpoints, use Phase 1F in `../DataSyncX_Refactor_Plan.md`.

```
I want to sync data with DataSyncX.

Source system/type: <source type, e.g. SQL Server / XEUS / Oracle>
Source object/query: <table, view, or SQL>

Target system/type: <target type, e.g. PostgreSQL / SQL Server>
Target table/schema: <target table>

Load mode: <full | incremental>
Incremental column/watermark: <column name or N/A>
Batch/range need: <none | by date | by id | custom>

Preferred build path: <direct demo script | demo_factory config/template>
Output location: <repo path for generated file>

Validation target: <unit only | local dry run | test DB run>
Constraints: <no hardcoded password, use get_config_value, etc.>

Please produce one bounded implementation path and the minimum validation gate.
```

Quick default version (minimum fields):

```
Sync from <source> to <target> using DataSyncX.
Mode: <full/incremental>, incremental key: <key or N/A>.
Use <direct demo script/demo_factory template> and create files under <path>.
```

Free-style quick-start version:

```
Build a free-style DataSyncX script from <source> to <target>.
Create the script at <path>. Use TODO placeholders for missing non-critical fields.
Then guide me one decision at a time to make it run-ready.
```

Expected agent behavior after you submit the form:

- If you choose `direct demo script`, the agent should create or update one runnable Python script in `src/datasyncx/demo/` (or your requested path), based on the nearest maintained demo pattern.
- If you choose `demo_factory config/template`, the agent should prefer config-driven artifacts under `src/datasyncx/demo_factory/` patterns and keep script generation template-first.
- If key fields are missing (for example incremental key or write mode), the agent should ask only the highest-leverage follow-up questions before generating files.

Additional behavior expectations:

- For free-style path, the agent should proactively include one optional performance topology recommendation (multi-thread batch read plus bounded single-queue write) without blocking script generation.
- If throughput or backpressure symptoms are later reported, the agent should escalate to stronger queue and flush tuning guidance.
- If requested reader, writer, or source-target pair is unsupported, the agent should ask approval before issue escalation, and if that path fails provide fallback contact guidance to damon.tang@intel.com.
- If target project venv readiness is unknown, the agent should run install preflight guidance: verify PyPI config state on Windows, guide permissioned system-level config updates when needed, and verify datasyncx version in the target venv.

One-Turn Workflow Contract

The default interaction pattern is:

1. Generate a bounded script/template path quickly.
2. Patch with one decision per turn.
3. Validate via manual-first command guidance.
4. Offer optional deployment guidance only after local acceptance.

First-Turn Examples

Use these short examples to set operator expectations.

1. If user says “from XEUS to SQL Server”

- Agent should pick one route, ask output path, and deliver one bounded artifact quickly.

1. If user says “help me build free-style”

- Agent should scaffold first, then ask one decision at a time.
- Agent should include optional lightweight performance topology advice immediately.

1. If user asks unsupported pair

- Agent should not pretend support exists.
- Agent should ask for approval before escalation and provide fallback contact guidance if escalation fails.

Operating Expectations

The agent is DataSyncX-first, but not framework-first at any cost. It should prefer existing DataSyncX abstractions when they fit, and it should keep product-specific predicates, schedules, target schemas, credentials, and business rules in product repositories.

When implementation is requested, expect the agent to keep the change bounded, identify risk level, update docs when needed, and report exact validation commands and results.

Runtime changes under `src/datasyncx/**` require targeted tests plus adjacent regression checks for the same public behavior.

The agent should ground itself in repository evidence before answering from memory. The expected evidence order is: `etl-agent-playbook.md`, the nearest usage/component docs, the nearest maintained demo, then the owning runtime code and tests.

For secrets handling, the agent should treat inline password literals as something to remove, not preserve. In DataSyncX code paths, the default expectation is to use `get_config_value(...)` or an existing reader/writer auto-resolution path when the component already derives the password from the configured account name.

For docs or customization-only changes, runtime tests are normally unnecessary. Validate agent frontmatter, links, documentation navigation, and changelog entries instead.

For install guidance, the agent should separate concerns clearly:

- Agent discovery comes from workspace files under `.github/agents/`.
- Runtime import and execution come from `datasyncx` installed in the target project venv.

Local Context Matters

The local VS Code workflow is the recommended way to use the agent because it can see repository files, tests, Git state, and local documentation. A web service can be useful for centralized review or non-VS-Code users, but it needs separate authentication, repository indexing, execution sandboxing, secrets boundaries, and audit controls before it can safely replace the local workflow.